Procesado y Análisis de Datos en Redes 5G (srsRAN)

Desarrollo de un *Log Parser* asíncrono con visualización en tiempo real

Manuel Calero Catena, José Miguel Romero Cascajo, Juan de Dios Sánchez Sanz

Asignatura: Laboratorio de Proyectos Tecnológicos (LPIT)

20 de abril de 2026

Índice

1. Introducción **2**

2. Marco Tecnológico **3**

2.1. Programación Asíncrona (Asyncio) 3

2.2. Polars vs Pandas 3

2.3. Dash de Plotly 3

3. Arquitectura del Sistema **4**

4. Desarrollo e Implementación **5**

4.1. Lectura en tiempo real (Tail -f) 6

4.2. Extracción de Volúmenes (Fase 1) 6

4.3. Correlación de Identificadores 5G (Fase 2) 6

5. Resultados y Visualización **7**

6. Conclusiones **8**

Resumen

El despliegue de redes 5G y el uso de plataformas Open Source como srsRAN y Open5GS generan grandes volúmenes de datos de telemetría y registros de red (logs). El procesamiento manual de estos archivos resulta inviable en entornos de producción. Esta memoria documenta de manera exhaustiva el desarrollo de una herramienta analítica implementada en Python, capaz de ingerir registros de red en tiempo real, procesar volúmenes de datos mediante `Polars`, correlacionar identificadores complejos del plano de usuario y de control de 5G, y proporcionar una visualización interactiva utilizando `Dash`. Adicionalmente, se garantiza la persistencia eficiente de los datos agregados mediante el formato `Parquet`.

1. Introducción

En el ámbito de las telecomunicaciones móviles modernas (4G/5G), la monitorización del estado de la red y del tráfico cursado por los usuarios (UE - *User Equipment*) es fundamental para garantizar la Calidad de Servicio (QoS). Con el auge de las implementaciones definidas por software (SDR), como srsRAN, se ha abierto la puerta a analizar internamente el comportamiento de los nodos, como la Unidad Central (CU) y la Unidad Distribuida (DU).

El objetivo principal de esta práctica es desarrollar un *script* que automatice el análisis de un fichero de logs generado por la CU, emulando un escenario de ingesta en vivo. Las metas de desarrollo se dividen en dos fases bien diferenciadas:

- **Fase 1 (Procesamiento de datos UP):** Localizar los mensajes que reportan transferencias de datos en *Downlink* a nivel de la subcapa SDAP, agregar en bloques de 1 segundo el volumen enviado y guardarlos en una estructura de datos de alto rendimiento. Además, se deben guardar periódicamente (cada 30 segundos) en un archivo Parquet.

- **Fase 2 (Identificación de CP):** Rastrear la creación inicial del UE en el plano de control para capturar parámetros de configuración clave, como el identificador RNTI, PCI y PLMN, logrando así asociar esta configuración estática a las transferencias dinámicas de datos calculadas en la fase anterior.

Ambas fases deben integrarse en un panel de control interactivo (*Dashboard*) que represente la evolución temporal del consumo de datos, todo ello operando en estricto tiempo real y sin bloqueos de ejecución.

2. Marco Tecnológico

El desarrollo se fundamenta en un ecosistema robusto de bibliotecas Python de última generación orientadas al rendimiento de datos.

2.1. Programación Asíncrona (Asyncio)

El proyecto requiere lidiar constantemente con operaciones de Entrada/Salida (lectura de archivos y servicio web). Emplear hilos estándar (*threads*) en Python conlleva un rendimiento mermado por el GIL (*Global Interpreter Lock*). Por el contrario, `asyncio` permite una concurrencia cooperativa eficiente en un solo hilo. Esto garantiza que mientras se espera a que se escriba una nueva línea en el log, la CPU puede dedicarse a empaquetar los datos para la interfaz gráfica.

2.2. Polars vs Pandas

Se ha descartado el uso de `Pandas` a favor de `Polars`. Desarrollada en Rust y fundamentada en Apache Arrow, `Polars` aplica una evaluación paralela (multinúcleo) y minimiza las copias de memoria en los DataFrames. Para operaciones de agrupación (*group_by*) repetitivas cada segundo, Polars reduce significativamente la latencia.

2.3. Dash de Plotly

Para la interfaz de visualización, se emplea `Dash`. Este entorno (*framework*) abstrae la necesidad de programar en JavaScript y React, permitiendo desarrollar interfaces reactivas completas exclusivamente con código Python, usando el componente `dcc.Interval` para consultas de actualización en vivo.

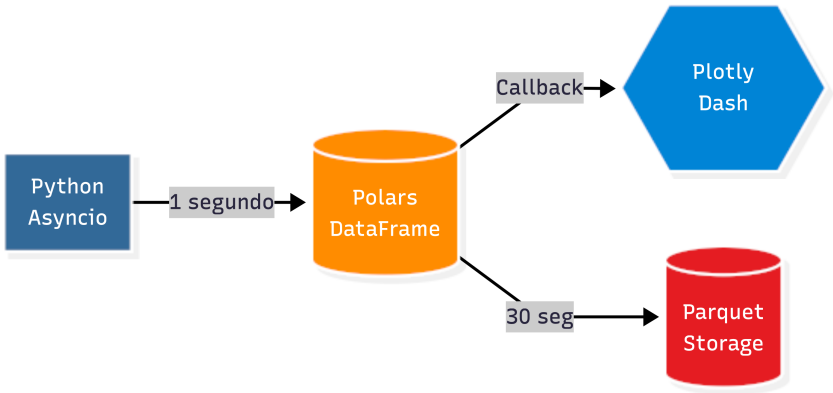

Figura 1: Esquema de tecnologías: Python, Polars, Dash, Parquet.

3. Arquitectura del Sistema

La estructura del código sigue el clásico patrón arquitectónico **Productor-Consumidor**, modificado para operar bajo un bucle de eventos asíncrono. Esta topología asegura la separación de responsabilidades (*Separation of Concerns*).

1. **Productor (`tail_file_producer`):** Analiza el final del archivo de texto, descarta líneas vacías, realiza coincidencias por expresiones regulares, acumula mediciones SDAP durante el segundo transcurrido y genera un micro-DataFrame. Al acabar, inyecta este DataFrame en una cola.

2. **Consumidor (`consumer`):** Aguarda mensajes en la cola. Tan pronto recibe el micro-DataFrame, realiza una concatenación vertical contra el DataFrame histórico global (`agg_df`).

3. **Dashboard (Dash Server):** Ejecutado en un entorno aislado (*thread*) para no bloquear a `asyncio`. Lee el DataFrame global y actualiza el cliente web.

4. **Persistencia (`parquet_saver`):** Una corrutina durmiente que, cada 30 segundos, ordena a Polars crear un archivo binario `data.parquet`.

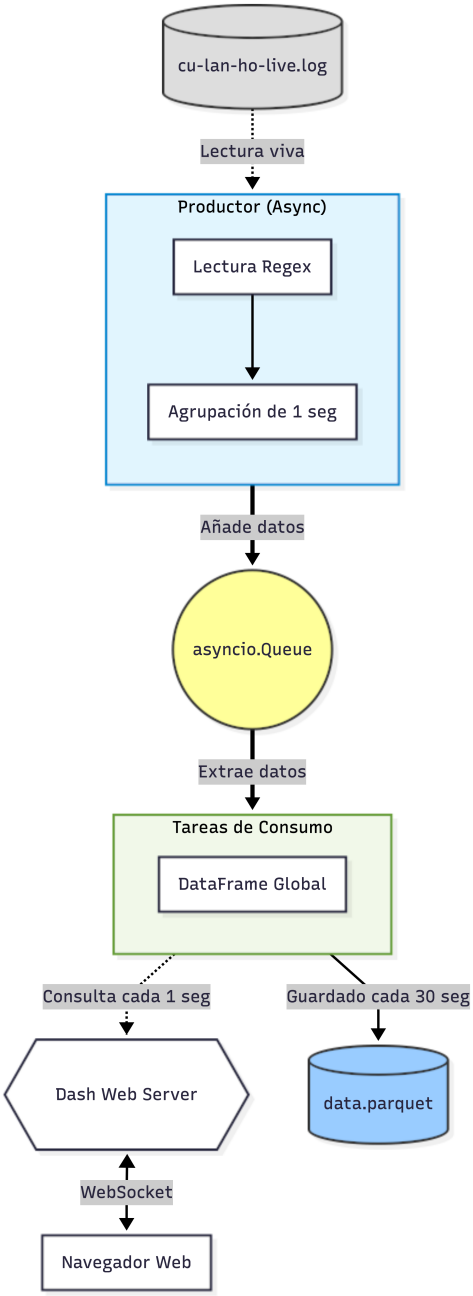

Figura 2: Diagrama de arquitectura Productor-Consumidor.

4. Desarrollo e Implementación

A continuación, se desgranan las soluciones algorítmicas empleadas para resolver los requisitos clave del laboratorio.

4.1. Lectura en tiempo real (Tail -f)

Para leer un archivo que está siendo activamente modificado por un proceso externo (`log-sim.py`), no se puede confiar en un simple bucle `for line in file`. Si se llega al final, el bucle terminaría y cerraría el archivo. La técnica correcta empleada implica almacenar el puntero en memoria (`f.tell()`) y utilizar un bucle infinito que aplique `seek()` para retroceder en caso de encontrar una línea parcial (sin salto de línea).

```
while True:
    pos = f.tell()
    line = f.readline()
    if not line or not line.endswith('\n'):
        f.seek(pos)
        await asyncio.sleep(0.1) # Ceder control a otras tareas
        continue
    # Procesamiento de la linea...
```

Listing 1: Lectura en tiempo real asíncrona

4.2. Extracción de Volúmenes (Fase 1)

La subcapa SDAP (*Service Data Adaptation Protocol*) de 5G mapea los flujos de QoS a los portadores de radio (Radio Bearers). En los logs, aparece bajo la firma `[SDAP ]`. Usando `re.compile`, buscamos el ID de usuario (`ue`) y el volumen transmitido (`pdu_len`):

```
re_sdap = re.compile(r'\[SDAP\s*\] \[D\] ue=(\d+).*?DL: TX PDU.*?pdu_len
    =(\d+)')
```

Se comprueba la estampa de tiempo (timestamp) de los 19 primeros caracteres de la línea (`2026-01-19T08:57:06`). Cuando este texto difiere del almacenado, se infiere que el segundo ha concluido. Entonces, el buffer acumulado pasa a `Polars` donde se suma agrupando por UE.

4.3. Correlación de Identificadores 5G (Fase 2)

Esta es la capa lógica más elaborada del proyecto. Las arquitecturas modernas desagregadas separan el procesamiento de señalización (Control Plane - CP) de los datos (User Plane - UP).

- La entidad **CU-CP** gestiona los parámetros de red: identificador de red (PLMN), celda física (PCI) y el identificador temporal de la radio (RNTI). Se registra como, por ejemplo, `ue=6`.
- La entidad **CU-UP** transmite el volumen y gestiona datos. Se registra con un contador independiente, por ejemplo, `ue=1`.

El log muestra un intercambio E1AP entre ambos para establecer el contexto de usuario (`cu_cp_ue`). Para reconstruir la relación, se diseñó una "máquina de estados.ªpoyada en diccionarios:

1. Se lee `[CU-UEMNG]` con `ue=6` → Guardamos metadatos en un diccionario con clave `6`.
2. Se lee `[CU-CP-E1]` → Descubrimos que el `ue=6` está asignado al contexto `cu_cp_ue=5`.
3. Se lee `[CU-UP-E1]` → Detectamos la entrada de la sesión `5`.
4. Seguidamente, se lee `[CU-UP] ue=1` → Sabemos que el `ue=1` es quien usó el contexto de la sesión `5`, que a su vez procedía del `ue=6`. Finalmente, asignamos los metadatos al usuario de datos `ue=1`.

Gracias a este mapeo a tres bandas, se extrae el contexto vital del usuario sin afectar al rendimiento del sistema.

```
2026-01-19T08:56:04.447149 [GTPU    ] [I] ue=0 UL teid=0x0088be: TX PDU. pdu_len=68 teid=0x0088be qfi=QFI=1
2026-01-19T08:56:04.447150 [PDCP    ] [D] ue=0 DRB2 UL: Removing sn=4923 from window
2026-01-19T08:56:04.447151 [PDCP    ] [D] ue=0 DRB2 UL: RX entity state. rx_next=4924 rx_deliv=4924 rx_reord=4922
2026-01-19T08:56:04.447154 [PDCP    ] [D] ue=0 DRB2 UL: Using sec_engine with worker_idx=3. count=4924 pdu_len=43
2026-01-19T08:56:04.447155 [PDCP    ] [D] ue=0 DRB2 UL: Security passed. count=4924
2026-01-19T08:56:04.447172 [PDCP    ] [D] ue=0 DRB2 UL: Adding sn=4924 to window.
2026-01-19T08:56:04.447175 [PDCP    ] [I] ue=0 DRB2 UL: RX SDU. count=4924
2026-01-19T08:56:04.447178 [SDAP    ] [D] ue=0 psi=2 QFI=1 DRB2 UL: RX SDU. QFI=1 sdu_len=40
2026-01-19T08:56:04.447185 [GTPU    ] [I] ue=0 UL teid=0x0088be: TX PDU. pdu_len=56 teid=0x0088be qfi=QFI=1
2026-01-19T08:56:04.447192 [PDCP    ] [D] ue=0 DRB2 UL: Removing sn=4924 from window
2026-01-19T08:56:04.447194 [PDCP    ] [D] ue=0 DRB2 UL: RX entity state. rx_next=4925 rx_deliv=4925 rx_reord=4922
2026-01-19T08:56:04.447315 [PDCP    ] [D] ue=1 SRB1 DL: Stopped PDCP entity
2026-01-19T08:56:04.447316 [PDCP    ] [D] ue=1 SRB1 UL: Stopped PDCP entity
2026-01-19T08:56:04.447441 [CU-CP-E1] [D] ue=1: UE context not found
2026-01-19T08:56:04.447445 [CU-CP-F1] [D] ue=1 cu_ue=0: Removing F1AP UE context
2026-01-19T08:56:04.447452 [NGAP    ] [D] ue=1 ran_ue=1 amf_ue=290: Removing NGAP UE context
2026-01-19T08:56:04.447472 [CU-UEMNG] [D] ue=1: Removed
2026-01-19T08:56:04.447474 [CU-CP   ] [I] ue=1: "UE Removal Routine" finalized
2026-01-19T08:56:04.447513 [NGAP    ] [I] Tx PDU ran_ue=1 amf_ue=290: UEContextReleaseComplete
```

Figura 3: Captura de los logs (plano de control vs plano de usuario).

5. Resultados y Visualización

Una vez ingeridos los datos, el servidor Dash ofrece una página local (en `127.0.0.1:8050`) que refresca un lienzo de `Plotly Graph Objects`. En el *Callback* principal, se itera sobre los valores únicos del Dataframe: `agg_df["ue"].unique()`. Por cada móvil (UE) distinto, se pinta una traza (línea) exclusiva.

La leyenda se enriquece accediendo al diccionario global de correlación. De este modo, en lugar de mostrar de forma aséptica ÜE 1", el interfaz aporta un valioso contexto operacional: ÜE 1 (PLMN: 21405 PCI: 501 RNTI: 0x4605)".

Durante la ejecución sobre el archivo de pruebas `cu-lan-ho.log`, la gráfica logra representar con claridad el proceso de **Handover**. Se observa cómo aparecen líneas de

volumen correspondientes a nuevos dispositivos, transmiten un alto grado de información, y finalizan su rastro cuando el equipo pasa a otra celda o pierde cobertura.

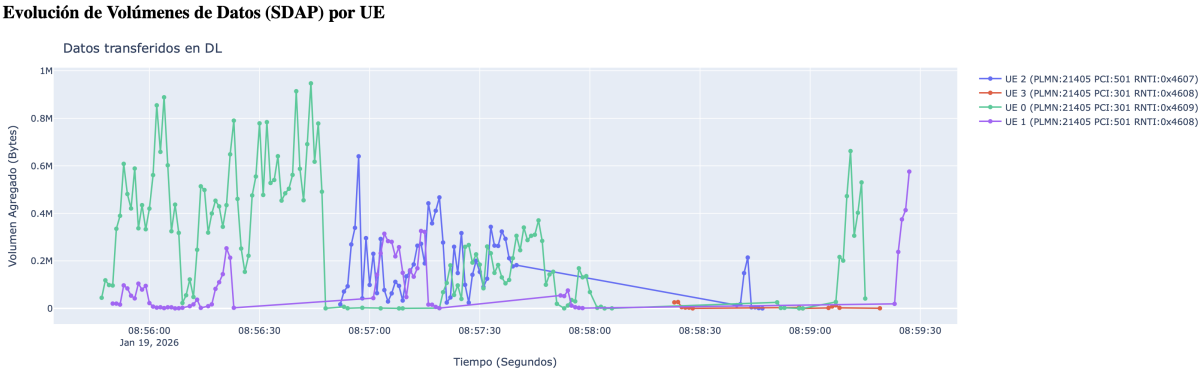

Figura 4: Captura del Dashboard de Plotly Dash en funcionamiento.

Por último, el requisito de persistencia de datos (Fase 1, punto 6) se comprueba examinando el directorio tras 30 segundos. Almacenarlos en formato Parquet significa que, a futuro, analistas de datos podrían importar este registro comprimido usando Apache Spark o herramientas Hadoop para realizar analítica masiva.

6. Conclusiones

El desarrollo final presentado en el script `log-parser.py` resuelve satisfactoriamente las carencias del análisis estático convencional de redes móviles. La implementación de la Práctica engloba conceptos altamente complejos de Ingeniería del Software y Telecomunicaciones:

- Tratamiento de datos mediante concurrencia asíncrona avanzada.
- Manejo de flujos (Streams) con diccionarios de correlación de estado.
- Interfaz gráfica reactiva sin el uso del navegador DOM manual.
- Comprensión de interfaces lógicas (E1AP) y separación CP/UP del 3GPP en 5G Standalone.

El código se mantiene a la vez legible (cumpliendo principios de la mentalidad Python o *Pythonic*) y altamente funcional, sentando las bases de lo que podría ser un servicio contenedorizado (Docker) para la recolección de estadísticas RAN en infraestructuras *Cloud-Native* como Open5GS y srsRAN.